Unit 11

1) Vertex cover: A vertex cover is a set of vertices such that every edge has atleast one endpoint in the set

Independent set: An independent set is a set of vertices such that no 2 vertices are adjacent (no edge b/w them)

clique: It is a set of vertices such that every pairs of vertices is connected by an edge.

Relationship: 1) vertex cover ↔ independent set

They are complements: If c is a vertex cover, then V-c is an independent set.

2) clique ↔ independent

A clique in a graph = independent set in its complement graph

Justification: 1) If c is a vertex cover then every edge has atleast one endpoint in c. So, no edge can have both endpoints in V-c

Hence, the vertices in V-c are not connected independent set

conversely: If I is an independent set then no edge inside I. So, every edge must connect to atleast 1 vertex in V-I. Hence V-I is a vertex cover

2) In a clique every pair of vertices has an edge in G. In $\bar{G}$ those edges do not exist.

So no 2 vertices are adjacent in $\bar{G}$ then independent set conversely is also true

2) a) polynomial time(P): It is the class of decision problems that can be solved in polynomial time by a deterministic algorithm.

b) non-deterministic polynomial time (NP): It is the class of decision problems for which a given solution can be verified in polynomial time not necessarily easy to solve, but easy to check.

c) NP-complete: A problem is NP-complete if

1. It is in NP &
2. Every problem in NP can be reduced to it in polynomial time. These are the hardest problems in NP

d) NP-hard: A problem is NP-hard if every NP problem can be reduced to it in polynomial time. It may or may not be in NP (verification may not be polynomial)

=> P ⊆ NP

NP-complete ⊆ NP-hard

NP-complete problems are both in NP & hardest

NP-hard problems are atleast as hard as NP problems.

3) a) Vertex cover

certificate: A set of vertices C

verification: check every edge has atleast 1 endpoint in C

GO through all edges → O(E)

polynomial => in NP

b) Independent set

certification: A set of vertices

verification: check no 2 vertices in S share an edge

check all pairs edges → O(E)

polynomial => in NP

c) clique

certificate: A set of vertices s

verification: check every pair of vertices in s has an edge

pairwise check → $O(|s|^2)$ => polynomial => in NP

d) Hamiltonian path

certificate: A sequence of vertices

verification: check each vertex appears once

checking consecutive vertices have edge → $O(V+E)$

polynomial => in NP

e) K-coloring ($K \geq 3$)

certificate: Assignment of colors to vertices

verification: for each edge, check endpoints have diff colors → $O(E)$ => polynomial => in NP

f) 3-SAT

certificate: Assignment of TRUE/FALSE to variables

verification: check each clause (3 literals) is satisfied → O(no. of clauses)

polynomial => in NP

4) Decision version of clique:

problem: Given a graph $G=(V,E)$ and an integer k

Does G contain a clique of size $\geq k$

step 1: clique ∈ NP

certificate: A set of k vertices

verification: check every pair has an edge → $O(k^2)$

polytime => clique ∈ NP

step 2: clique is NP-Hard (reduction from 3-SAT)

we reduce 3-SAT to clique

construction: Given a 3-SAT formula with m clauses: $c_1, c_2, \ldots, c_m$.

each clause has 3 literals

Build graph G: 1. vertices: create one vertex for each literal in each clause → total 3m vertices

2. edges: connect vertices from diff clauses only
   Add edge if literals are not contradictory (not x & ¬x)

3. set: k = m

Correctness proof:

*If 3-SAT is satisfiable
choose one true literal from each clause
These m literals are: from different clauses
Not contradictory
so all are connected then form a clique of size m

* If clique of size m exists, clique must pick 1 vertex per clause. All vertices are mutually connects then no contradictions
Assign truth values accordingly
This satisfies all clauses ⇒ 3-SAT is satisfiable

conclusion of reduction: Reduction is polynomial
3-SAT ≤p clique ⇒ clique is NP-hard

** Reduction: Hamiltonian path then hamiltonian cycle

Idea: convert a graph G into a new graph G' s.t G has a hamiltonian path iff G' has a hamiltonian cycle.

Construction: Given G = (V, E)

1. Add a new vertex $v_{new}$
2. connect $v_{new}$ to all vertices in V.

Correctness: If G has a hamiltonian path

suppose path is $v_1 \to v_2 \to \cdots \to v_n$

Then in G':

$v_{new} \to v_1 \to \cdots \to v_n \to v_{new}$ forms cycle

If G' has a H cycle remove $v_{new}$ from the cycle then remaining is a Hamiltonian path in G.

Complexity: Adding 1 vertex and edges → O(V) (poly time)

b) Reduction: Hamiltonian cycle → Hamiltonian path

Idea: Transform graph so that finding a H. path corresponds to H. cycle

Construction: Given $G = (V, E)$

1. pick any vertex $v \in V$
2. Replace v with 2 vertices $v_1$ & $v_2$
3. For every edge $(u, v)$ belongs to 'E'.
   add edges $(v_1, u)$ & $(v_2, u)$
4. Do not connect $v_1$ & $v_2$

Correctness:

* If G has a H cycle

  break the cycle at v → get a path

  Replace v with $v_1$ & $v_2$ → gives a H. path in G'.

* If G' has a H. path

  Path must start at one of $v_1$, $v_2$ & end at the other

  Merge them back into v → forms a H. cycle in G

Complexity: Vertex split + edge updates → O(V+E) poly time.

6) Travelling salesman problem (TSP):

a) Optimization version

Find a tour visiting all cities exactly once with min total cost

b) Decision version

Given a graph with costs and bound B→

Is there a tour listing all vertices with cost ≤ B?

TSP ∈ NP

certificate: a tour

verification: Verify cost ≤ B in polynomial time

Hence, in NP

NP-completeness (reduction from H-cycle)

construction: Make graph complete,

Assign weights: edge in G → 1

not in G → 2

Set B = |V|

correctness: If H-cycle exists → cost = |V| → yes

If tour ≤ |V| → all edges cost 1 → valid H-cycle

∴ TSP (decision) is NP-complete.

7) a) weighted Independent set (Tree)

Idea: For each vertex v, compute

Dp[v][1]: max wt. including v

Dp[v][0]: max wt. excluding v.

Recurrence: If v is included: $Dp[v][1] = w(v) + \sum_{u \in child(v)} Dp[u][0]$

If v is excluded: $Dp[v][0] = \sum_{v \in child(v)} \max(Dp[v][0], Dp[v][1])$

Ans: max ( dp[root][0], dp[root][1])

complexity : O(V)

b) weighted vertex cover (True)

If v is included

$$dp[v][1] = w(v) + \sum_{u \in children(v)} \min(dp[u][0], dp[u][1])$$

If v is excluded

$$dp[v][0] = \sum_{u \in children(v)} dp[u][1]$$

Ans: min ( dp[root][0], dp[root][1])

complexity : O(V)